UTEC POSTGRADO

CURSO

“BASE DE DATOS”

Duolingo - Plataforma Educativa

(parte 2)

Integrantes del grupo 5:

Cristopher Hurtado

Alvaro Colquicocha

Ruben Coquil

Juan Guido Silva

Alex Junior Valdez

Enrique Huasupoma

Lima, 15 de abril de 2026

1. Caso de uso de Ciencia de Datos

Para Duolingo proponemos un sistema de **aprendizaje personalizado basado en errores de estudiante**. El objetivo es detectar patrones de error (p. ej. confusión de tiempos verbales, género, sintaxis) en las respuestas de cada alumno y recomendar ejercicios de refuerzo específicos para superar esos errores. Cada vez que un estudiante completa o falla un ejercicio, se genera un evento (ID de usuario, ejercicio, habilidad, resultado, timestamp, etc.). Con millones de interacciones diarias, Duolingo ya procesa enormes volúmenes de datos (decenas de millones de usuarios diarios), por lo que un enfoque tradicional de SQL no escala ni maneja bien la variedad y semántica de estos datos.

Este caso de uso genera datos de alta variedad y velocidad: registros semi-estructurados (JSON) de cada intento, texto de respuestas, metadata de habilidades lingüísticas, y potencialmente audio. Además, se usarán técnicas de IA para convertir estos datos en vectores de características (embeddings) que capturan la similitud semántica entre el perfil del estudiante, los tipos de error y los ejercicios disponibles. Estas cargas de datos (logs de eventos, texto libre, vectores numéricos) encajan mejor en arquitecturas modernas (no relacionales y de streaming) que en el modelo relacional rígido del primer trabajo.

Duolingo, como líder global en este sector, ha cimentado su éxito en una infraestructura de datos masiva que procesa aproximadamente mil millones de ejercicios diarios realizados por más de 600 millones de usuarios registrados. No obstante, el desafío contemporáneo no reside únicamente en la escala de la operación, sino en la profundidad cualitativa de la personalización. La presente propuesta técnica detalla una arquitectura de ciencia de datos diseñada para trascender los modelos probabilísticos tradicionales, implementando un sistema de refuerzo adaptativo basado en el análisis vectorial de errores mediante la integración de Databricks Lakehouse y DynamoDB.

1.1. Limitaciones del modelo actual:

El núcleo pedagógico de Duolingo se ha fundamentado históricamente en dos algoritmos propietarios: Birdbrain y la Regresión de Vida Media (HLR). Birdbrain utiliza principios de la Teoría de Respuesta al Ítem (IRT) para estimar simultáneamente la dificultad de un ejercicio y la competencia del estudiante, actualizando estos valores tras cada interacción para mantener al usuario en una zona de desafío óptima o "estado de flujo". Por otro lado, el modelo HLR predice la probabilidad de recuerdo de una palabra específica basándose en el historial de práctica y el tiempo transcurrido, optimizando la repetición espaciada.

A pesar de la eficacia demostrada de estos modelos, los cuales han impulsado un crecimiento del 67% en los usuarios activos mensuales y una mejora del 17% en las tasas de éxito de las lecciones, existe una oportunidad latente para profundizar en la naturaleza del error. Actualmente, el sistema detecta si una respuesta es correcta o incorrecta, pero a menudo carece de una comprensión semántica del "por qué" del fallo. Un error puede ser el resultado de un desliz ortográfico (typo), una confusión fonética, una interferencia de la lengua materna o una falta de comprensión profunda de una regla gramatical compleja.

El caso de uso que proponemos, denominado **"Refuerzo Cognitivo Semántico"**, busca

transformar cada error cometido por el estudiante en un vector de características en un espacio latente de alta dimensionalidad. En lugar de simplemente marcar una respuesta como incorrecta, el sistema analiza la distancia semántica entre la respuesta del usuario y la respuesta correcta. Si un estudiante confunde el verbo "ser" con "estar" en español, el sistema debe identificar que el error es de naturaleza semántico-gramatical y no meramente ortográfico. Mediante el uso de embeddings vectoriales, la plataforma puede buscar en su vasto catálogo de contenidos aquellos ejercicios que ataquen específicamente la brecha de conocimiento revelada por ese error particular.

Componente del Modelo	**Enfoque Tradicional (IRT/HLR)**	**Propuesta de Análisis Vectorial**
Definición del Error	Binaria (Correcto/Incorrecto)	Vectorial (Matices semánticos y fonéticos)
Selección de Refuerzo	Basada en dificultad y etiquetas manuales	Basada en similitud de coseno en espacio latente
Perfil del Usuario	Estimación escalar de competencia	Perfil Dinámico de Estudiante (SPV) multidimensional
Feedback	Genérico basado en reglas predefinida	RAG (Retrieval-Augmented Generation) contextual

Este enfoque permite que la plataforma actúe con precisión. Por ejemplo, en ejercicios de pronunciación, la inteligencia artificial puede utilizar modelos de procesamiento de lenguaje natural (NLP) y reconocimiento de voz (ASR) para detectar si el error se debe a un problema de acentuación o a una mala articulación de un fonema específico. Al vectorizar estos errores fonéticos, el sistema puede recomendar ejercicios que se centren exclusivamente en los sonidos que presentan mayor dificultad para ese usuario específico, optimizando el tiempo de estudio y reduciendo la frustración.

2. Selección y justificación de la tecnología NoSQL

2.1. Selección: MongoDB+Databricks

Se propone una arquitectura basada en una base de datos documental **MongoDB Atlas** junto con **Databricks** para el procesamiento de datos y la generación de embeddings. MongoDB permite el almacenamiento de documentos JSON flexibles y, mediante Atlas Vector Search, ofrece un índice vectorial incorporado para realizar búsquedas por similitud semántica. Por su parte, Databricks aporta un lakehouse unificado optimizado para procesos ETL, streaming y entrenamiento de modelos de machine learning a gran escala.

Aunque la solución ideal considera MongoDB Atlas con capacidades vectoriales como núcleo operativo, **para la implementación práctica en el entorno educativo (AWS Academy) se utilizará Amazon DynamoDB como base NoSQL operativa**. Esta decisión responde a restricciones de acceso y costo, manteniendo el mismo modelo conceptual de documentos y delegando las capacidades de búsqueda semántica y procesamiento vectorial a Databricks.

MongoDB permite esquemas dinámicos, escalado horizontal y consultas ricas sobre documentos (por ejemplo, perfiles de usuario con listas de errores) sin la rigidez del modelo relacional. Además, Atlas Search soporta índices vectoriales (embeddings) junto a datos operativos, lo que resulta ideal para recuperar ejercicios semánticamente relacionados con el perfil de aprendizaje del estudiante.

Databricks, por su parte, facilita la orquestación de flujos de datos en capas (arquitectura Medallion: Bronze, Silver y Gold), la ejecución de procesos de limpieza mediante Spark Streaming y el escalado del entrenamiento de modelos de recomendación utilizando clusters distribuidos.

Para implementar una solución de análisis vectorial a la escala de Duolingo, se requiere una infraestructura capaz de procesar millones de interacciones en tiempo real, ofreciendo respuestas en milisegundos a usuarios concurrentes. La combinación de MongoDB Atlas (como referencia arquitectónica ideal) y Databricks se justifica por la convergencia de flexibilidad, escalabilidad y capacidades avanzadas de análisis semántico.

2.2. Justificación de la selección de MongoDB Atlas

A diferencia de las bases de datos relacionales tradicionales que requieren esquemas rígidos, MongoDB Atlas utiliza un modelo de documentos flexible que permite almacenar datos semi-estructurados de forma natural. En el contexto de Duolingo, esto es vital porque los diferentes tipos de ejercicios (traducción, dictado, conversación, ejercicios de opción múltiple) tienen estructuras de datos divergentes. MongoDB permite que todos estos tipos de contenido residan en una sola colección de "ejercicios", facilitando consultas complejas sin la necesidad de uniones de tablas costosas.

La justificación técnica de MongoDB Atlas sobre otras soluciones NoSQL o bases de datos vectoriales puras (como Pinecone o Milvus) reside en la eliminación del "impuesto de sincronización". En arquitecturas tradicionales, el desarrollador debe sincronizar constantemente una base de datos operativa con una base de datos vectorial externa, lo que introduce latencia, complejidad en el código y posibles inconsistencias de datos. MongoDB Atlas Vector Search integra las capacidades de búsqueda semántica directamente en la base de datos principal, permitiendo que los embeddings vectoriales se almacenen junto con los metadatos operativos (como el nivel del curso, el idioma y la tasa de éxito histórica) en el

mismo documento

Criterio de Selección	Beneficio para Duolingo	Justificación Técnica
Modelo Multimodelo	Manejo de texto, metadatos y vectores en una sola plataforma	Reduce el TCO (Total Cost of Ownership) y la complejidad arquitectónica
Sincronización Nativa	Los vectores se actualizan automáticamente con los cambios en los documentos	Garantiza que el refuerzo se base siempre en el historial más reciente del usuario.
Escalabilidad Global	Distribución de datos en múltiples regiones geográficas	Latencia mínima para usuarios en mercados emergentes como Asia y África
Búsqueda Híbrida	Combina similitud vectorial con filtros de palabras clave y metadatos	Permite filtrar ejercicios similares por nivel CEFR o unidad específica de gramática

2.3. Justificación de la selección de Databricks

Mientras MongoDB Atlas gestiona la operación en tiempo real, Databricks proporciona el entorno de procesamiento masivo necesario para la generación y el entrenamiento de los modelos de embeddings. La arquitectura Lakehouse de Databricks permite unificar los silos de datos, facilitando que los ingenieros de aprendizaje automático y los científicos de datos trabajen sobre una "única fuente de verdad". El uso de Apache Spark dentro de Databricks permite la vectorización masiva de todo el catálogo de Duolingo en minutos, una tarea que sería inabordable con herramientas convencionales

La capacidad de Databricks para gestionar el ciclo de vida completo del aprendizaje automático mediante MLflow y Unity Catalog asegura que cada versión del modelo de análisis de errores sea rastreable y gobernable. Esto es crítico para mantener la calidad pedagógica, ya que permite realizar pruebas A/B de diferentes modelos de embeddings (por ejemplo, comparando modelos basados en Transformers contra modelos de n-gramas de caracteres para detectar errores ortográficos específicos) y medir su impacto directo en la retención del usuario

2.4. Limitaciones del modelo relacional

El modelo relacional tradicional presenta diversas limitaciones para el caso de uso propuesto, el cual involucra procesamiento de texto, personalización y análisis semántico:

- ➔ Rigidez del esquema → no permite la evolución dinámica de tipos de error (gramaticales, fonéticos, semánticos), los cuales pueden cambiar con el tiempo y variar entre usuarios.
- ➔ No soporta embeddings eficientemente → los vectores de alta dimensionalidad utilizados para representar errores y ejercicios no se manejan de forma nativa en SQL, lo que dificulta su almacenamiento y consulta.
- ➔ JOINs costosos para perfiles complejos → la construcción de un perfil de estudiante requeriría múltiples uniones entre tablas (usuarios, ejercicios, errores, historial), generando alto costo computacional y latencia.
- ➔ No maneja bien texto no estructurado → las respuestas de los usuarios son texto libre, lo que complica su modelado en tablas tradicionales y limita su análisis semántico.
- ➔ No permite búsqueda semántica → SQL tradicional solo permite búsquedas exactas o por patrones, pero no puede identificar similitud conceptual entre errores y ejercicios, lo cual es clave para la recomendación personalizada.

En conjunto, estas limitaciones hacen que el modelo relacional no sea adecuado para implementar un sistema de aprendizaje adaptativo basado en análisis semántico y embeddings, como el propuesto en este proyecto.

3. Diseño del modelo de datos NoSQL

Se propone un diseño basado en colecciones que representan los dominios principales del sistema: estudiantes, ejercicios e intentos. El modelo combina datos operacionales con capacidades de análisis vectorial, permitiendo personalización en tiempo real.

3.1. Colección: students (Perfil dinámico del estudiante)

Este documento representa un Student Profile Vector (SPV), es decir, un perfil dinámico que evoluciona con cada interacción.

```
{
  "_id": "user_9988",
  "identidad": {
    "nombre": "Juan",
    "idioma_base": "es",
    "idioma_objetivo": "en",
```

```
  "nivel_cefr": "A2",
  "plan": "Super",
  "racha_actual": 45
},
"competencias": {
  "gramatica": 0.7,
  "vocabulario": 0.4,
  "pronunciacion": 0.6
},
"historial_reciente": [
  {
    "exercise_id": "ex1",
    "correcto": false,
    "tipo_error": "conjugation",
    "timestamp": "2026-04-12T10:05:00Z"
  },
  {
    "exercise_id": "ex2",
    "correcto": true,
    "tipo_error": null,
    "timestamp": "2026-04-12T10:10:00Z"
  }
],
"error_profile": [
  {
    "tipo_error": "conjugation",
    "conteo": 15,
    "ultimo": "2026-04-12",
    "embedding": [0.12, 0.23, ...]
  },
  {
    "tipo_error": "gender",
    "conteo": 7,
    "ultimo": "2026-04-10",
    "embedding": [0.05, 0.31, ...]
  }
],
"vector_debilidad": [0.05, -0.08, 0.72, ...],
"embedding_global": [0.512, 0.117, 0.904, ...],
"preferencias": {
  "temas_interes": ["viajes", "tecnologia"],
  "estilo_aprendizaje": "visual_auditivo"
},
"recomendaciones": ["ex5", "ex12"],
"ultima_actualizacion": "2026-04-12T10:15:00Z"
    }
```

3.2. Colección: exercises (Catálogo de ejercicios vectorizado)

Representa el contenido pedagógico enriquecido con embeddings.


```
{
  "_id": "ex_10452",
  "idioma_base": "es",
  "idioma_objetivo": "fr",
  "nivel_cefr": "A2",
  "unidad": "comida_y_restaurantes",
  "tipo_ejercicio": "traduccion_inversa",
  "contenido": {
    "texto": "Je voudrais un café, s'il vous plaît",
    "audio_uri": "s3://duo-content/audio/fr_cafe_01.mp3",
    "gramatica_foco": ["condicional", "cortesia"]
  },
  "habilidad": "gramatica",
  "metadata_dificultad": {
    "birdbrain_beta": 0.45,
    "tasa_exito_global": 0.82
  },
  "content_embedding": [0.124, -0.056, 0.892, ..., 0.031],
  "version_modelo": "gte-large-v1.5",
  "fecha_creacion": "2026-01-10"
}
```


Se recomienda crear un índice vectorial sobre:

1. content_embedding

3.3. Colección: attempts (Eventos de interacción)

Registra cada intento del usuario. Es la fuente de verdad para la analítica.

A continuación, se muestra un ejemplo simplificado de un evento de interacción de un usuario, el cual representa la unidad básica de datos del sistema:

Ejemplo de evento de interacción (formato JSON)

```
{
  "student_id": "u1",
  "input_text": "La casa es bonito",
  "correct_text": "La casa es bonita",
  "error_type": "gender",
  "recommended_exercises": [
    "La mesa es ____",
    "La silla es ____"
  ]
}
```

Este modelo simplificado permite comprender de forma intuitiva la lógica del sistema. En una implementación real, el documento se enriquece con información adicional, como se muestra a continuación:


```
{
  "_id": "att_778899",
  "student_id": "user_9988",
  "exercise_id": "ex_10452",
  "timestamp": "2026-04-12T10:05:00Z",
  "correcto": false,
  "answer_text": "Je quiero un cafe",
  "error_type": "code_switching",
  "metadata": {
    "tiempo_respuesta_seg": 12,
```

"dispositivo": "mobile",
"session_id": "sess_4455"
}
}

Este tipo de registro corresponde conceptualmente a la colección "attempts", la cual almacena cada interacción del estudiante con la plataforma y sirve como base para la analítica y personalización.

Se recomienda crear índices sobre campos clave como:

1. error_type
2. timestamp

3.4. Características del modelo NoSQL propuesto:

Flexibilidad:

1. Permite agregar nuevos tipos de error sin modificar el esquema
2. Soporta datos semiestructurados (texto, audio, vectores)

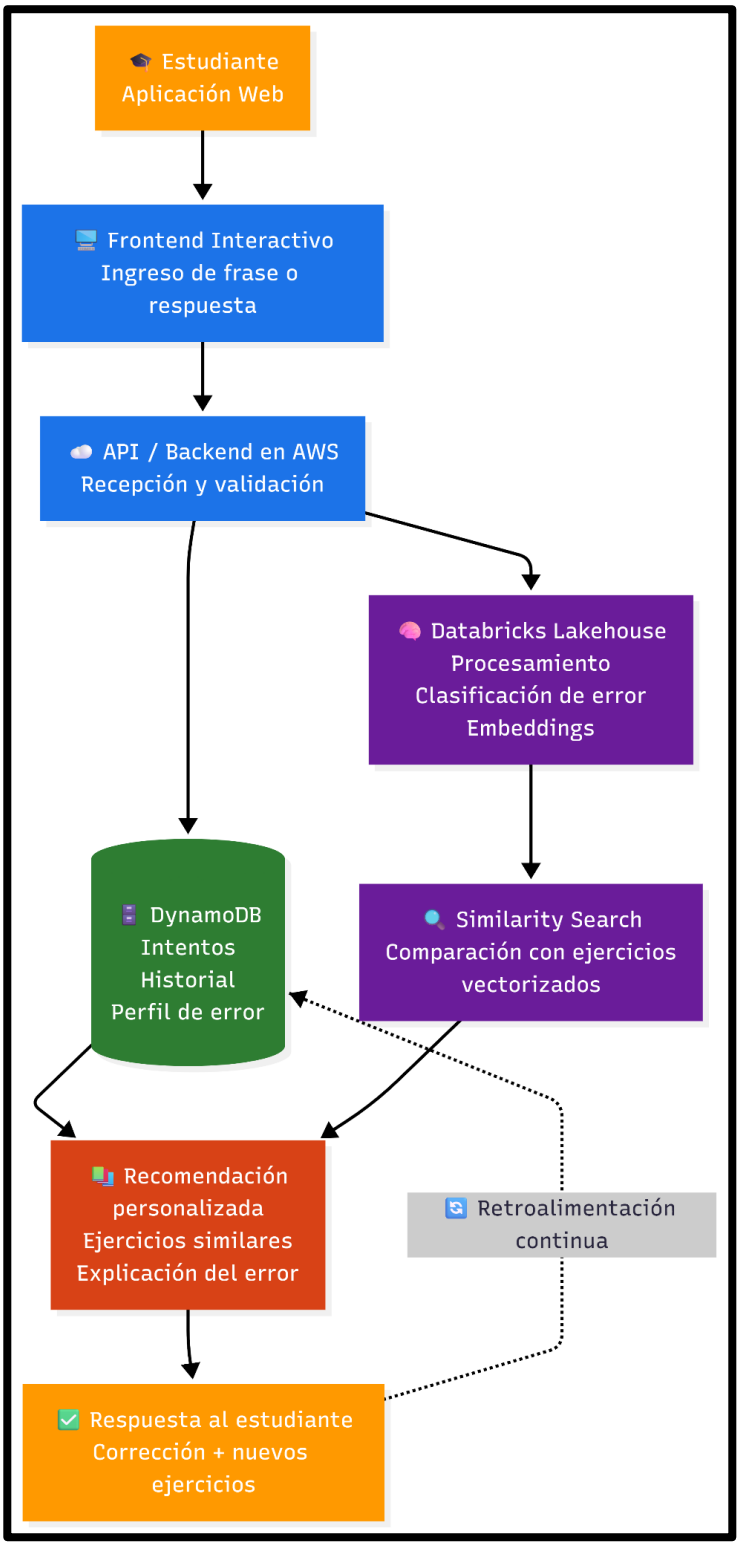
Patrones aplicados:

1. Atributos dinámicos: error_profile, competencias
2. Valores computados: vector_debilidad, embedding_global
3. Referencia extendida: inclusión de metadata del ejercicio en eventos

4. Aplicación y arquitectura de la solución

La solución se diseña como un ecosistema moderno de datos que integra procesamiento analítico distribuido, almacenamiento flexible y capacidades avanzadas de inteligencia artificial. Este enfoque combina el uso de Databricks Lakehouse como plataforma central de procesamiento con MongoDB Atlas como base de datos operacional, incorporando además búsqueda vectorial para habilitar recomendaciones personalizadas en tiempo real. La arquitectura adopta el patrón Medallion, lo que permite transformar datos crudos en información de alto valor de forma progresiva, manteniendo calidad, trazabilidad y escalabilidad. Adicionalmente, se incorpora una capa de servicios de baja latencia que permite consumir las recomendaciones directamente desde la aplicación.

Diagrama de arquitectura de la solución



La arquitectura propuesta integra una capa operativa en AWS con una capa analítica en Databricks. DynamoDB almacena los eventos de interacción y el historial del estudiante, mientras que Databricks procesa los datos, genera embeddings y permite la recomendación personalizada de ejercicios mediante similitud semántica.

Resumen del flujo de trabajo

- El estudiante ingresa una frase o respuesta en la plataforma.
- El frontend envía la interacción al backend desplegado en AWS.
- El backend registra el intento en DynamoDB como evento de interacción.
- Databricks procesa la respuesta del estudiante para identificar el tipo de error.
- A partir del error detectado, se genera una representación semántica o embedding del caso.
- El sistema compara ese error con ejercicios previamente vectorizados o clasificados.
- Se seleccionan ejercicios de refuerzo adaptados a la debilidad detectada.
- La recomendación generada se devuelve a la aplicación.
- El estudiante recibe corrección, explicación y nuevos ejercicios personalizados.
- Cada nueva interacción retroalimenta el perfil del estudiante, mejorando la personalización futura.

4.1. Ingesta de Datos y Generación de Eventos

El flujo inicia con la generación continua de eventos por parte de los usuarios. Cada interacción dentro de la plataforma, como responder un ejercicio, cometer un error, escribir una oración o interactuar con contenido de audio produce datos en tiempo real que incluyen identificadores de usuario, ejercicio, resultado, tipo de error, tiempo de respuesta y otros metadatos relevantes.

Estos eventos son capturados mediante tecnologías de streaming como Apache Kafka o Amazon Kinesis, lo que permite manejar grandes volúmenes de información con alta velocidad y baja latencia. Este enfoque es necesario debido a la escala de Duolingo, donde millones de interacciones deben ser procesadas simultáneamente sin degradar el rendimiento del sistema.

4.2. El Pipeline de Datos Medallion en Databricks

La ingesta y transformación de datos sigue un flujo de refinamiento progresivo

- **Capa Bronze (Ingesta Raw):** En esta capa se almacenan los datos tal como llegan desde las fuentes, sin transformaciones significativas. Los eventos se guardan en tablas Delta Lake, lo que permite mantener un historial completo para auditoría y reprocesamiento. Esta capa actúa como la fuente de verdad del sistema, asegurando que todos los datos estén disponibles para futuras transformaciones o análisis.
- **Capa Silver (Validación y Enriquecimiento):** En la capa Silver los datos son depurados y estandarizados. Se eliminan duplicados, se manejan valores nulos y se normalizan formatos de fechas e identificadores. Además, se incorporan procesos de

enriquecimiento mediante modelos de procesamiento de lenguaje natural (NLP), los cuales permiten clasificar los errores cometidos por los estudiantes en categorías como gramaticales, léxicas, sintácticas o fonéticas. Esta capa es crítica porque transforma datos operacionales en información semántica, permitiendo entender no solo si un estudiante falló, sino la naturaleza específica de su error.

- **Capa Gold (Analítica y AI-Ready):** Se crean las tablas finales que alimentan los perfiles de usuario y el catálogo de ejercicios. Estas tablas están optimizadas para el rendimiento y la gobernanza, sirviendo como base para el entrenamiento de modelos de predicción de abandono (churn) y para la actualización masiva de la base de datos operativa MongoDB Atlas

En la capa Gold los datos se consolidan y se preparan para consumo analítico y modelos de machine learning. Se generan características relevantes como métricas de desempeño por habilidad, frecuencia de errores y patrones de comportamiento.

En esta etapa también se generan embeddings mediante modelos de aprendizaje automático. Un enfoque común es el modelo de dos torres (two-tower), donde una red neuronal aprende representaciones vectoriales de los estudiantes y otra de los ejercicios. Estos embeddings capturan relaciones semánticas complejas y son fundamentales para el sistema de recomendación.

4.3. Generación de Embeddings y Representación Vectorial

Los embeddings representan el núcleo del sistema. Estos vectores de alta dimensionalidad codifican información sobre estudiantes, ejercicios y errores en un espacio semántico común.

Por ejemplo, un error de conjugación verbal tendrá una representación vectorial cercana a ejercicios que refuercen ese mismo concepto. De esta forma, el sistema puede identificar similitudes no evidentes mediante reglas tradicionales. Los embeddings se generan en Databricks y se almacenan en MongoDB o en índices de búsqueda vectorial para su posterior consulta.

4.4. Lógica de Recomendación Basada en Similitud

El sistema de recomendación funciona mediante comparación de vectores. Cuando se necesita sugerir contenido, se obtiene el embedding del estudiante o de su perfil de error y se compara con los embeddings de los ejercicios disponibles.

La similitud se calcula mediante métricas como cosine similarity o producto punto, lo que permite identificar los ejercicios más cercanos en el espacio vectorial. El sistema selecciona los Top-K ejercicios más relevantes, logrando una personalización mucho más precisa que los enfoques basados en reglas o segmentación simple.

4.5. Flujo de Trabajo del Refuerzo Adaptativo

El sistema opera en un ciclo de retroalimentación continua:

1. **Generación de Eventos:** El estudiante falla un ejercicio de traducción. La aplicación envía la respuesta incorrecta y el ID del ejercicio a MongoDB.
2. **Detección de Anomalías y Clasificación:** Un trigger de MongoDB Atlas detecta la inserción del error y puede invocar opcionalmente una función de AWS Lambda o una función nativa de Atlas para una respuesta inmediata. Simultáneamente, el evento fluye hacia Databricks mediante el conector de Spark para un análisis profundo
3. **Inferencia Vectorial:** En Databricks, el modelo de IA convierte el error en un vector. Si el error es "¿Ella comer manzanas?" en lugar de "¿Ella come manzanas?", el vector captura la falta de concordancia sujeto-verbo.
4. **Actualización del Perfil de Usuario:** El SPV del usuario se actualiza en MongoDB. Sus "coordenadas de conocimiento" se desplazan ligeramente hacia la zona de debilidad de "conjugación en presente".
5. **Búsqueda Semántica de Refuerzo:** Cuando el usuario solicita una lección de repaso, la aplicación ejecuta una agregación $vectorSearch en la colección de ejercicios de MongoDB:
   JavaScript
   ```
   db.ejercicios.aggregate()
   ```
6. **Entrega de Contenido:** El usuario recibe ejercicios que son semánticamente relevantes para su error pero estructuralmente diferentes, lo que refuerza la plasticidad neuronal y evita la memorización de respuestas por repetición monótona

4.6. Integración de RAG para Explicaciones Pedagógicas

La arquitectura incorpora técnicas de Retrieval-Augmented Generation (RAG) para mejorar la calidad de las explicaciones al usuario. En este enfoque, cuando un estudiante solicita entender su error, el sistema primero recupera información relevante —como reglas gramaticales o ejemplos— mediante búsqueda vectorial.

Este contexto es enviado a un modelo de lenguaje que genera una explicación precisa y contextualizada. A diferencia de un modelo generativo puro, este enfoque reduce errores y mejora la coherencia pedagógica, ya que las respuestas están basadas en contenido previamente validado.

4.7. MLOps y Gobernanza de Datos

La solución incorpora prácticas rigurosas de MLOps. Databricks Vector Search permite la sincronización automática (Delta Sync), lo que significa que cualquier actualización en el catálogo de cursos en el Lakehouse se refleja casi instantáneamente en el índice de búsqueda de la aplicación sin intervención manual. La gobernanza se garantiza mediante Unity Catalog, permitiendo el control de acceso a nivel de fila y columna, asegurando que los datos de los estudiantes se manejen de acuerdo con las regulaciones vigentes

Capa de Arquitectura	Tecnología	Función Crítica
Almacenamiento Operativo	MongoDB Atlas	Persistencia de perfiles, sesiones y vectores de búsqueda.
Motor de Búsqueda Semántica	Atlas Vector Search	Recuperación de ejercicios de refuerzo en < 20ms.
Procesamiento de Datos	Databricks Lakehouse	Ingesta de telemetría y ejecución de pipelines Medallion.
Inteligencia Artificial	PySpark / MLflow / LLMs	Generación de embeddings y orquestación de RAG.
Gobernanza y Seguridad	Unity Catalog / MongoDB IAM	Gestión de identidades, cifrado y auditoría de datos.

5. Conclusiones y Recomendaciones Estratégicas

La arquitectura propuesta representa un salto cualitativo en la personalización educativa. Al integrar MongoDB Atlas y Databricks, Duolingo puede transformar el error de una métrica de fracaso a un sensor de aprendizaje de alta fidelidad. Esta solución no solo mejora la eficiencia del aprendizaje mediante el refuerzo adaptativo vectorial, sino que también optimiza los costos operativos al consolidar las necesidades operativas y analíticas en una plataforma de datos unificada y escalable.

Se recomienda a la organización:

1. **Priorizar la transición a vectores multimodales:** No solo texto, sino también vectores de audio para mejorar la retroalimentación en ejercicios de pronunciación, donde los errores fonéticos son los más difíciles de corregir sin intervención humana
2. **Monitorear el "Flow" del Estudiante:** Utilizar las métricas de Databricks para ajustar dinámicamente los parámetros de búsqueda vectorial, asegurando que el contenido de refuerzo mantenga siempre el equilibrio perfecto entre desafío y maestría
3. **Expandir el uso de Digital Twins de Estudiantes (DTS):** Evolucionar el SPV hacia una representación holística que incluya metas de carrera y preferencias de compromiso, permitiendo una orquestación de rutas de aprendizaje que trascienda la

mera gramática

En definitiva, la convergencia de la flexibilidad del documento NoSQL con la potencia de la analítica en la nube permite que Duolingo continúe liderando la revolución EdTech, haciendo realidad la promesa de una educación personalizada, efectiva y verdaderamente universal.